

Smart_Bulb_Lab

Hands-on: Smart Bulb Security Demonstration and Basic Testing

Learning Objectives

After completing this activity, students will be able to:

- Understand basic IoT device functionality
- Identify common IoT security vulnerabilities
- Execute basic security tests
- Observe real-time attack impacts
- Understand the importance of IoT security

Required Materials

- Smart Bulb with Linux (Ubuntu 24.04)
 - Computer with Windows OS
 - Internet connection
 - Web browser
 - Command Prompt/PowerShell
 - Text editor (Notepad or similar)
-

Instructions

1. 環境準備(Linux)

- 從 github 下載：

```
$ git clone https://github.com/gbox-ntut/smartbulb-lab.git  
$ cd smartbulb-lab
```

- 安裝執行環境：

```
$ python -m venv venv  
$ source venv/bin/activate  
$ pip install flask flask-socketio
```

2. 啟動Smart Bulb (Linux)

```
$ python smart_bulb_server.py
```

```
(venv) user@UB2604SRV:~/smartbulb-lab$ python smart_bulb_server.py
Smart Bulb Simulator Starting
Device ID: SB2000-0bc74d69
Web interface: http://localhost:8080
* Serving Flask app 'smart_bulb_server'
* Debug mode: on
INFO:werkzeug:WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
* Running on all addresses (0.0.0.0)
* Running on http://127.0.0.1:8080
* Running on http://127.0.0.1:8080
INFO:werkzeug:Press http://127.0.0.1:8080
INFO:werkzeug: * 按住 Ctrl 再按一下滑鼠左鍵，以追蹤連結
Smart Bulb Simulator
Device ID: SB2000-0bc74d69
Web interface: http://localhost:8080
WARNING:werkzeug: * Debugger is active!
INFO:werkzeug: * Debugger PIN: 779-021-724
INFO:werkzeug:192.168.56.1 - - [19/Sep/2026 11:21:27] "GET /socket.io/?EIO=4&transport=polling&t=Q2vPb1q HTTP/1.1" 200 -
INFO:werkzeug:192.168.56.1 - - [19/Sep/2026 11:21:27] "POST /socket.io/?EIO=4&transport=polling&t=Q2vPbJw&sid=mqdBj7B8JzPr3ojwAAAA HTTP/1.1" 200 -
INFO:werkzeug:192.168.56.1 - - [19/Sep/2026 11:21:27] "GET /socket.io/?EIO=4&transport=polling&t=Q2vPbJ_&sid=mqdBj7B8JzPr3ojwAAAA HTTP/1.1" 200 -
```

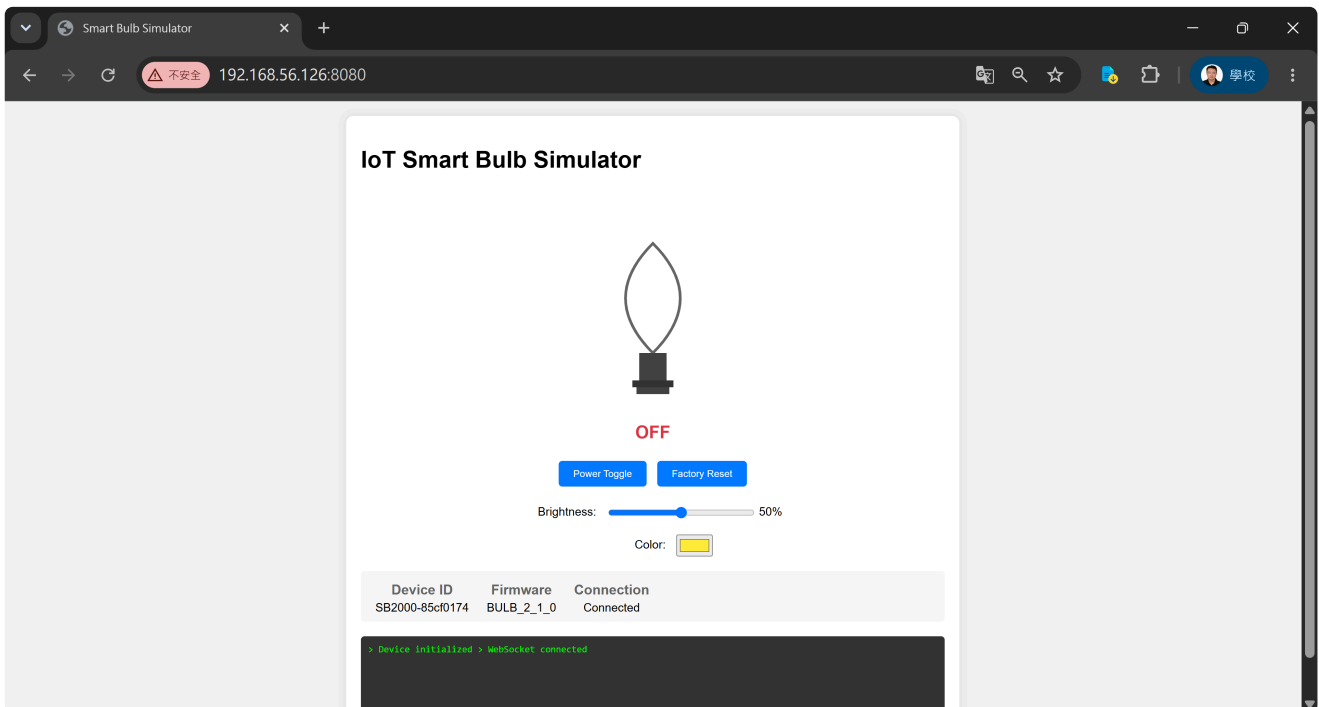
- 確認已有開啟8080 port

```
$ netstat -an|grep '8080'
```

```
tcp        0      0 0.0.0.0:8080        0.0.0.0:*           LISTEN
tcp        0      0 192.168.56.126:8080 192.168.56.1:2009   ESTABLISHED
```

3. 存取Smart Bulb (Linux)

- 開啟瀏覽器存取 <http://192.168.56.126:8080/>



基本介面測試

- 點選電源按鈕開啟/關閉燈泡
- 移動亮度滑桿
- 使用顏色選擇器變更顏色
- 記錄顯示的設備資訊
- 記錄觀察結果

您能看到哪些關於該設備的資訊？

- 有哪些可用的控制選項？
- 存取該設備是否需要密碼？

API Endpoints

API (Application Programming Interface) 是一組規則、協定和工具，它允許軟體應用程式彼此通訊。它充當不同軟體系統之間的橋樑，使它們能夠以標準化的方式共享資料或功能。

- GET / # Web interface
- POST /api/control/power # Power control
- POST /api/control/brightness # Brightness control
- POST /api/control/color # Color control
- POST /api/control/factory_reset # Factory reset

4. 使用curl 確認設備狀態

```
c:\> curl http://<IP>:8080/api/status
```

```
C:\>curl http://192.168.56.126:8080/api/status
{
  "brightness": 50,
  "color": "#FFEB3B",
  "device_id": "SB2000-85cf0174",
  "firmware_version": "BULB_2_1_0",
  "power": false
}
```

※ 記錄收到的訊息。

※ 注意任何洩漏的敏感資料。

5. 控制測試

- (1). 開啟燈

```
curl -X POST http://<IP>:8080/api/control/power -H "Content-Type: application/json" -d "{\"state\": true}"
```

```
C:\>curl -X POST http://192.168.56.126:8080/api/control/power -H "Content-Type: application/json" -d "{\"state\":true}"
{
  "message": "Power ON",
  "state": {
    "brightness": 50,
    "color": "#FFEB3B",
    "device_id": "SB2000-85cf0174",
    "firmware_version": "BULB_2_1_0",
    "power": true
  },
  "success": true
}
```

- (2). 關閉燈

```
curl -X POST http://<IP>:8080/api/control/power -H "Content-Type: application/json" -d "{\"state\": false}"
```

※ 命令執行成功了嗎？

※ 網頁介面發生了什麼變化？

- (3). 開到最大亮度

```
curl -X POST http://<IP>:8080/api/control/brightness -H "Content-Type: application/json" -d "{\"brightness\": 100}"
```

- (4). 變更顏色

```
curl -X POST http://<IP>:8080/api/control/color -H "Content-Type: application/json" -d "{\"color\": \"#FF0000\"}"
```

6. 小結

完成這項物聯網安全實踐實驗室活動後，學生們透過模擬智慧燈泡系統，獲得了分析和測試物聯網設備安全性的實務經驗。透過這次互動課程，他們學會了識別和利用物聯網設備中常見的安全漏洞，同時了解了這些安全缺陷在現實世界中可能造成的影響。
